



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Instituto de Ciências Exatas e de Informática

Integração de Single Sign-On em Microfrontends: Desafios e Estratégias com Vite e Module Federation*

Single Sign-On Integration in Microfrontends: Challenges and Strategies with Vite and Module Federation

Caroline Clara de Souza¹

Wesley Dias Maciel²

Resumo

O aumento da complexidade em aplicações *front-end* tem intensificado os desafios relacionados à modularidade, escalabilidade e segurança. Neste contexto, a arquitetura de *Microfrontends* propõe a divisão da interface em domínios independentes, promovendo maior autonomia entre equipes e facilitando a manutenção. A integração da autenticação *Single Sign-On* em ambientes distribuídos, contudo, representa um desafio relevante, especialmente quanto ao compartilhamento seguro de sessões entre módulos isolados. Este trabalho tem como objetivo investigar estratégias para essa integração, utilizando as tecnologias Vue.js, Vite e *Module Federation*. Para tanto, foi desenvolvida uma aplicação *showcase* que simula cenários reais de autenticação e comunicação entre MFE. A metodologia empregada baseia-se na experimentação prática e na análise qualitativa dos aspectos de desempenho, segurança e complexidade da solução. Os resultados demonstram que o uso de *cookies* HTTP proporciona uma alternativa segura para manter sessões entre domínios, e que a composição dinâmica via *Module Federation* é viável, embora demande conhecimento técnico avançado. Conclui-se que a arquitetura proposta é eficiente e pode servir de referência para implementações similares em projetos *web* modernos.

Palavras-chave: microfrontends; autenticação; single sign-on; vite; module federation; vue.js.

*Artigo apresentado ao Instituto de Ciências Exatas e Informática da Pontifícia Universidade Católica de Minas Gerais como pré-requisito para obtenção do título de Bacharel em Sistemas de Informação.

¹Aluna do Programa de Graduação em Sistemas de Informação, Brasil – carolineclaradsl@gmail.com.

²Professor do Programa de Graduação em Ciência da Computação, Brasil – wesleydiasmaciel@gmail.com.

Abstract

The increasing complexity of front-end applications has intensified challenges related to modularity, scalability, and security. In this context, the Microfrontend architecture proposes splitting the interface into independent domains, promoting greater team autonomy and facilitating maintenance. However, integrating Single Sign-On authentication into distributed environments represents a significant challenge, especially regarding the secure sharing of sessions between isolated modules. This study aims to investigate strategies for this integration using Vue.js, Vite, and Module Federation technologies. To achieve this, a *showcase* application was developed to simulate real authentication and communication scenarios between Microfrontends. The methodology adopted is based on practical experimentation and a qualitative analysis of the solution's performance, security, and complexity aspects. The results demonstrate that the use of HTTP cookies offers a secure alternative for maintaining sessions across domains, and that dynamic composition via Module Federation is feasible, despite requiring advanced technical knowledge. It is concluded that the proposed architecture is efficient and can serve as a reference for similar implementations in modern web projects.

Keywords: microfrontends; authentication; single sign-on; vite; module federation; vue.js.

1 INTRODUÇÃO

Nos últimos anos, a tendência de incorporar cada vez mais lógica ao *front-end* ganhou força. Os usuários passaram a exigir interfaces mais interativas e serviços de maior qualidade (MEZZALIRA, 2021). Entretanto, os desafios aumentam à medida que aplicações *front-end* monolíticas (POWELL e SMALLEY, 2024) crescem com a adição de novas funcionalidades, melhorias e correções. O crescimento da base de código dificulta a leitura e o entendimento por parte dos desenvolvedores, tornando a manutenção mais complexa e aumentando o risco de introdução de novos erros (GEERS, 2020).

Para lidar com esse cenário, a arquitetura de *Microfrontends* (MFE) pode ser aplicada como uma solução potencial. Os MFE se caracterizam como uma abordagem que promove a divisão do projeto em domínios, permitindo que as equipes tenham maior autonomia na definição de prioridades e na escolha das estratégias de implementação (GEERS, 2020).

Entretanto, assim como qualquer padrão arquitetural, MFE apresentam desafios de implementação. Entre esses desafios, a comunicação se destaca como uma das principais decisões a serem tomadas (MEZZALIRA, 2020), uma vez que diferentes MFE podem precisar compartilhar informações. Um exemplo crítico dessa necessidade ocorre na autenticação de usuários em uma aplicação *web*, pois é imprescindível garantir um meio seguro para o compartilhamento de *tokens* entre os diferentes MFE. O controle de acesso desempenha um papel indispensável na mitigação de ataques contra a segurança de aplicações *web* (BOONKRONG, 2020).

Diante desse cenário, este estudo tem como objetivo geral desenvolver uma aplicação *showcase* (MFESHOWCASE, 2025) que exemplifique a autenticação, a integração e o funcionamento de MFE, permitindo a análise das melhores práticas e dos desafios encontrados durante o desenvolvimento. Especificamente, busca-se explorar a utilização do *Module Federation* (WEBPACK, 2020) para a composição de MFE na ferramenta de empacotamento Vite (VITE, 2025), além de investigar a viabilidade da implementação do método de autenticação única *Single Sign-On* (SSO) (SCAPICCHIO; FORREST, 2024) em ambientes baseados em MFE. Como base tecnológica, o projeto utiliza o *framework* JavaScript Vue.js, amplamente adotado no desenvolvimento de aplicações *front-end* (VUE, 2021).

A justificativa para a realização deste estudo fundamenta-se na necessidade de compreender, de forma prática, como o *Module Federation* pode ser aplicado em conjunto com o Vite para viabilizar a construção de *Microfrontends* que atendam a requisitos de autenticação única por meio do *Single Sign-On*, contribuindo para o entendimento de seus benefícios e desafios.

A construção da aplicação viabilizou a avaliação da implementação da autenticação por SSO, além de possibilitar a identificação das principais dificuldades e facilidades associadas ao uso do *Module Federation* no Vite. Adicionalmente, foi realizada uma análise de desempenho e segurança, proporcionando uma visão abrangente dos impactos da arquitetura escolhida. A

principal contribuição da pesquisa consiste em apresentar um padrão para modularização e autenticação no desenvolvimento de aplicações *front-end*.

Este trabalho é direcionado a desenvolvedores *front-end*, arquitetos de *software* e equipes técnicas envolvidas na construção de aplicações *web* escaláveis e modulares. O público-alvo abrange profissionais interessados na adoção da arquitetura de *Microfrontends*, especialmente em contextos que utilizam Vue.js e Vite, e que necessitam validar a possibilidade de autenticação por meio de SSO. Além disso, a solução proposta é relevante para organizações que buscam aumentar a autonomia entre as equipes de desenvolvimento, sem comprometer a segurança e a coesão da aplicação.

2 REFERÊNCIAL TEÓRICO

Neste capítulo, são apresentados os conceitos fundamentais, tecnologias e abordagens utilizadas no desenvolvimento da aplicação proposta. Serão explorados os princípios da arquitetura de MFE, o funcionamento da abordagem *Module Federation* dentro do ecossistema Vite, conceitos de autenticação por SSO, *cookies* HTTP e *Core Web Vitals*. Além disso, serão discutidos trabalhos relacionados que contribuem para a fundamentação teórica deste estudo.

2.1 Fundamentação Teórica

2.1.1 Arquitetura de *Microfrontends*

A arquitetura de Microserviços (MS) emergiu como uma estratégia para lidar com a alta complexidade e os desafios de manutenção de sistemas monolíticos. Inicialmente aplicada em projetos de *back-end*, projetos que seguem o padrão de MS são compostos por pequenos serviços independentes e intercomunicáveis, modelados em torno de um domínio de negócio, seguindo princípios do *Domain-Driven Design* (DDD) (KHONONOV, 2021) que auxilia a construção de *softwares* alinhados a um domínio de negócio. Com essa arquitetura torna-se mais evidente onde cada funcionalidade deve ser implementada, permitindo que equipes de *back-end* sejam organizadas de forma autônoma, cada uma responsável por um ou mais serviços específicos, o que reduz a complexidade individual de cada módulo (NEWMAN, 2015).

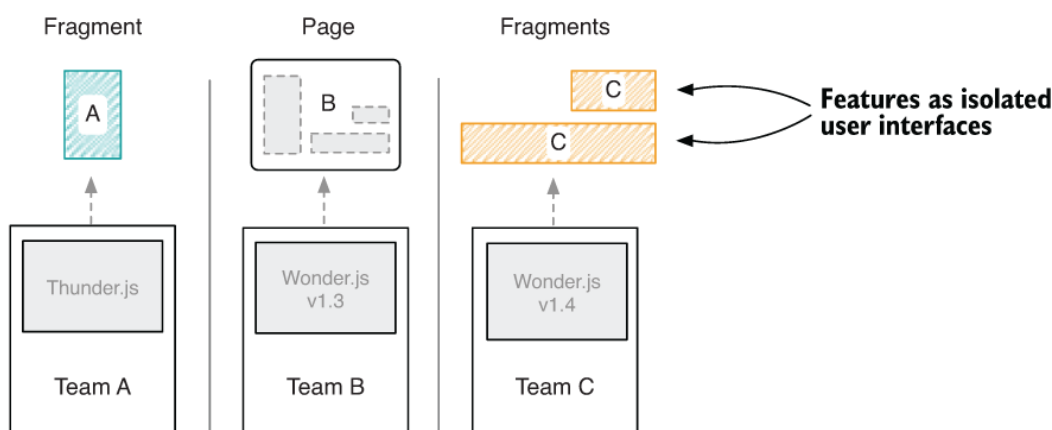
Dada a semelhança entre os desafios enfrentados por grandes projetos monolitos (POWELL e SMALLEY, 2024) no *back-end* e no *front-end*, conceitos consolidados dos MS foram adaptados para o *front-end*, dando origem ao padrão arquitetural conhecido como *Microfrontends*. Esse padrão arquitetural divide a aplicação *front-end* em fatias verticais. As fatias são interfaces de usuário (*User Interface* – UI) construídas a partir de dados retornados por serviços, bancos de dados ou MS. Cada fatia é de responsabilidade de uma equipe e são integradas no navegador web do cliente a fim de formar a página final (GEERS, 2020).

A adoção de MFE pode auxiliar na redução do tempo de desenvolvimento, pois, com a divisão do projeto em domínios, as equipes ganham maior autonomia para a definição de prioridades e poder de escolha sobre como será a implementação. Além disso, cada equipe pode

definir o prazo de entrega do artefato ao usuário sem impactar as demais, tornando esse processo mais descentralizado (GEERS, 2020).

Por trás de cada fatia há um time responsável pela entrega de um fragmento ou página que irá compor a aplicação final (Figura 01). Essas interfaces de usuário são desenvolvidas e funcionam de forma isolada, dessa forma, caso alguma delas estiver fora do ar, nada impedirá que as outras funcionem aplicadas em outros contextos. Cada equipe responsável por um MFE tem o poder de decidir a tecnologia e qual versão desta tecnologia melhor atenderá a suas demandas e, por se tratar de aplicações que não compartilham código, *frameworks* e bibliotecas JavaScript, é possível garantir que o funcionamento em conjunto não será impactado (GEERS, 2020).

Figura 1 – Divisão de aplicações MFE entre times *front-end*, que podem ser responsáveis por páginas ou fragmentos.



Fonte: Geers (2020).

Na divisão por páginas, uma página inteira é de responsabilidade de uma equipe dentro do time de *front-end*. A equipe deve manter esta página em funcionamento e desenvolver ou acrescentar as funcionalidades e componentes que a constituem. Cada página é fracamente acoplada e podem ser acessíveis apenas por meio de um domínio público (GEERS, 2020).

Quando há um componente que aparece repetidamente dentro de várias páginas de uma aplicação MFE, não é ideal que ele seja refeito onde for necessário novamente. Nesses casos, a solução seria a criação de apenas um componente para evitar a replicação de código. Esse componente pode ser entendido como um fragmento. Os fragmentos são mini aplicativos incorporáveis e isolados do resto da página, onde equipes (responsáveis por outras páginas) podem utilizar um fragmento desenvolvido por outra time sem a necessidade de conhecer seus detalhes de implementação (GEERS, 2020).

Assim como qualquer padrão arquitetural, a abordagem de MFE apresenta desafios de implementação. Definir o modo de visualização — se haverá múltiplos MFE carregados simultaneamente ou apenas um por vez —, a composição, o roteamento e a estrutura de

comunicação são decisões arquitetonicas que devem ser tomadas antecipadamente, pois impactam diretamente o futuro do sistema (MEZZALIRA, 2021).

2.1.1.1 Roteamento e Transições de Páginas

A integração por roteamento e transição de páginas se concentra na navegação do usuário entre páginas desenvolvidas por diferentes equipes. Nesse tipo de incorporação de MFE, é possível utilizar *Hiperlinks* ou *Application Shell* como técnicas de integração.

Os *Hiperlinks* representam a forma mais simples de integrar páginas MFE, pois uma equipe precisa apenas inserir um link para acessar o conteúdo desenvolvido por outra (GEERS, 2020). No entanto, essa abordagem geralmente exige chamadas a um servidor para recuperar as páginas, caracterizando-se como uma integração *server-side*.

O *Application Shell* pode ser definido como um componente base contendo o mínimo necessário de HTML, CSS e JavaScript para fornecer a interface de usuário de uma aplicação. Essa técnica é frequentemente utilizada em *Progressive Web Apps* (PWA), onde o template base é armazenado em cache *offline*, melhorando o desempenho do carregamento do aplicativo (OSMANI, 2020). No contexto de MFE, o *Application Shell* permite a transição entre páginas no lado do cliente (*client-side*), atuando como uma aplicação central que determina qual página deve ser exibida com base na URL atual. Isso implica que, ao alterar a URL, a página correspondente também é modificada (GEERS, 2020).

2.1.1.2 Composição

A composição é utilizada quando se deseja incorporar fragmentos em páginas. Para isso, existem três abordagens principais: integração *server-side*, integração *client-side* e integração híbrida.

Na integração *server-side*, a composição dos fragmentos ocorre antes que a página seja exibida no navegador do cliente. Um servidor compositor é responsável por montar a página final, atuando como intermediário entre o navegador e os servidores que fornecem as páginas e fragmentos desenvolvidos pelas equipes de *front-end*. Como alternativa ao uso de um servidor compositor, a equipe responsável por uma página pode buscar diretamente os fragmentos fornecidos por outras equipes (GEERS, 2020).

Já na integração *client-side*, cada fragmento funciona como um pequeno aplicativo independente, e a composição ocorre diretamente no navegador, utilizando a API *Document Object Model* (DOM). Essa abordagem permite que os fragmentos sejam renderizados e atualizados sem impactar o restante da página. Além disso, quando necessário, os fragmentos podem se comunicar com outros MFE por meio da transmissão de eventos. Dentro desse contexto, os *Web Components* surgem como um modelo padronizado que permite encapsular fragmentos independentes, desenvolvidos em diferentes tecnologias, garantindo sua coexistência em uma mesma página (GEERS, 2020).

Por fim, a integração híbrida combina diferentes estratégias para compor fragmentos em páginas, sendo os *iframes* e chamadas Ajax duas das técnicas mais comuns. Os *iframes*

proporcionam isolamento entre os fragmentos, garantindo um fraco acoplamento, de modo que qualquer alteração no conteúdo dentro do *iframe* não afete o restante da página. Já o Ajax permite uma integração mais dinâmica, onde chamadas JavaScript são feitas ao servidor para carregar fragmentos na página de forma assíncrona.

2.1.1.3 Comunicação

A comunicação em MFE diz respeito à necessidade de integração de informações entre diferentes fragmentos da aplicação. Para que os MFE possam interagir entre si, existem dois cenários principais de comunicação: interface de usuário e contexto global.

A comunicação por interface de usuário ocorre diretamente no navegador do usuário e envolve a propagação de dados entre diferentes elementos da aplicação, como página-fragmento, fragmento-página e fragmento-fragmento. Esse tipo de comunicação permite que os MFE atualizem seus estados e atributos conforme necessário. Nessa abordagem, cada equipe desenvolve seu MFE de forma a emitir mudanças de estado ou atributos, permitindo que outros MFE dependentes sejam atualizados adequadamente (GEERS, 2020).

Já a comunicação por contexto global surge da necessidade de manter todos os MFE sincronizados com informações compartilhadas. Para que isso seja possível, elas são armazenadas em um local centralizado, onde podem ser acessadas por diferentes fragmentos da aplicação. No entanto, esse contexto é geralmente restrito apenas à leitura (GEERS, 2020).

2.1.1.4 Princípios da Arquitetura de Microfrontends

A adoção da arquitetura de MFE demanda a consideração de princípios fundamentais (Quadro 1), que são, em grande parte, semelhantes aos aplicados em arquiteturas de MS.

Quadro 1 – Princípios da arquitetura de MFE.

Princípio	Descrição
Modelagem entorno de domínios de negócios	Dividir a aplicação em domínios para auxiliar na formação de equipes autônomas, especializadas e responsáveis por um ou mais domínios.
Cultura de automação	Garantir a construção de boas pipelines de integração e implantação contínuas com feedbacks precisos.
Ocultar detalhes de implementação	Fornecer contratos bem definidos para auxiliar na independência das equipes.
Descentralização sobre centralização	Permitir que as equipes possam tomar as decisões de implementação dentro do seu domínio especializado.
Implantar de forma independente	Reduzir as dependências externas para garantir sua atuação independente.
Isolar falhas	Impedir que MFE com problemas não impactem a aplicação final ao ocultar ou fornecer conteúdos alternativos.
Altamente observável	Usar ferramentas de observabilidade para auxiliar no monitoramento do estado da aplicação pelos desenvolvedores.

Fonte: adaptado de Mezzalana (2025).

É importante enfatizar que MFE aplicam-se para cenários específicos e devem ser adotados apenas quando seus pontos positivos superarem os negativos. Afinal, nenhuma solução deve ser considerada uma “bala de prata” para os desafios do sistema (BROOKS, 1995). Dessa forma, a tecnologia deve se adaptar ao problema, e não o contrário.

2.1.2 Module Federation

A arquitetura de MFE depende de técnicas de roteamento e composição para que cada domínio seja integrado à aplicação final. A introdução do *Module Federation* no Webpack 5 se tornou mais uma estratégia de composição. O *Module Federation* segue o princípio de unir compilações de diferentes aplicações em único aplicativo. Uma das principais vantagens dessa abordagem é possibilitar o compartilhamento de código e dependências entre diferentes compilações de aplicações, o que permite impulsionar a modularização e escalonamento (WEBPACK, 2020).

2.1.2.1 Module Federation no Ecossistema Vite

Baseando-se nos conceitos de *Module Federation* do Webpack 5, o *plugin vite-plugin-federation* (VITEFEDERATION, 2025) viabilizou a construção de MFE no ambiente de desenvolvimento do Vite. Entretanto, apesar das semelhanças conceituais, o *Module Federation* no Vite apresenta peculiaridades. Entre elas, destaca-se a definição de que sistemas que adotam essa abordagem são compostos de, no mínimo, dois projetos: aplicação hospedeira e aplicação remota (VITEFEDERATION, 2025).

Na configuração do módulo remoto, é possível expor seus recursos funcionais e elementos de interface. Já na configuração do módulo hospedeiro, são definidas quais aplicações remotas que fazem parte do ecossistema e que irão compor o aplicativo final. Além disso, é possível especificar quais dependências serão compartilhadas, permitindo que sejam resolvidas em tempo de compilação (VITEFEDERATION, 2025).

2.1.3 Single Sign-On

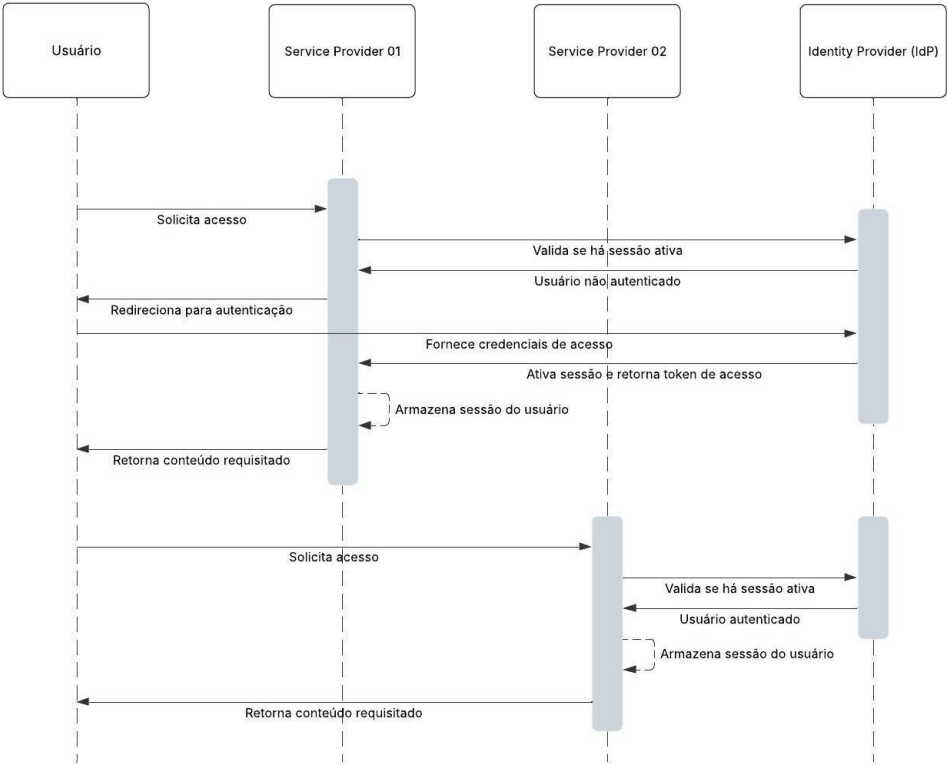
O *Single Sign-On* é um método de autenticação única que permite ao usuário acesso a várias aplicações utilizando uma única credencial. Esse processo ocorre por meio do compartilhamento da mesma sessão (SCAPICCHIO; FORREST, 2024).

O funcionamento do SSO baseia-se em uma relação de confiança mútua entre um ou mais *Service Providers* (SPs) e um *Identity Provider* (IdP). Quando um usuário fornece credenciais válidas, o IdP gera um *token* de acesso contendo informações sobre sua identidade. Assim, sempre que o usuário tentar acessar um SP dentro desse ecossistema de confiança, o SP confirma a autenticidade do *token* junto ao SSO. Caso o *token* seja considerado válido, é gerado um certificado digital que concede ao usuário acesso à aplicação (SCAPICCHIO; FORREST, 2024).

O fluxo fundamental de autenticação baseada em SSO consiste em cada SP dentro do sistema, ao receber uma requisição de acesso, verifica se há uma sessão ativa do solicitante.

Caso não haja sessão válida, o fluxo de autenticação é iniciado. Após uma validação ou autenticação bem-sucedida, caso ainda não tenha feito, o SP armazena a sessão do usuário. Dessa forma, em qualquer tentativa futura de acesso, o solicitante poderá ter acesso direto ao conteúdo sem a necessidade de uma nova autenticação (Diagrama 1).

Diagrama 1 – Fluxo básico de funcionamento de um sistema SSO.



Fonte: elaborado pelos autores com dados extraídos de Scapicchio; Forrest (2024).

2.1.4 Cookies HTTP

Cookies HTTP são pequenos arquivos de texto enviados por um servidor *web* para o navegador do usuário com o objetivo de armazenar informações que podem ser reutilizadas em requisições subsequentes. Sua principal função é possibilitar que o servidor mantenha o estado da comunicação com o usuário, uma vez que o protocolo HTTP (*Hypertext Transfer Protocol*) é por natureza sem estado (HTTP, 2025). Esses dados, denominados *cookies*, são geralmente utilizados para gerenciar sessões de usuário, personalizar a experiência do usuário no site e viabilizar funcionalidades como a manutenção do *login* e preferências de idioma (HTTPCOOKIES, 2025).

2.1.4 Core Web Vitals

O *Core Web Vitals* é um conjunto de métricas definidas pela Google como fundamentais para avaliar a experiência do usuário em páginas *web*. Essas métricas possibilitam que proprietários e desenvolvedores de sites aprimorem o desempenho de suas páginas ao focar nos aspectos mais relevantes de forma simplificada. Cada métrica representa uma dimensão

específica da experiência do usuário, sendo elas: carregamento (*Largest Contentful Paint* — LCP), interatividade (*Interaction to Next Paint* — INP) e estabilidade visual (*Cumulative Layout Shift* — CLS) (WALTON, 2020).

Para que um site seja considerado performático, ele deve atender a critérios elaborados pela Google (Quadro 2). Esses critérios são: (i) o LCP deve ocorrer em menos de 2,5 segundos; (ii) o INP deve estar disponível em menos de 200 milissegundos; e (iii) o CLS deve manter uma pontuação igual ou inferior a 0,1. Para ser aprovada, a página precisa atingir pelo menos 75% de conformidade em cada uma dessas métricas do *Core Web Vitals* (WALTON, 2020).

Quadro 2 – Categorização do *Core Web Vitals*.

	Bom	Precisa de melhorias	Ruim
LCP	2,5 segundos ou menos	Entre 2,5 segundos e 4,0 segundos	Acima de 4,0 segundos
INP	200 milissegundos ou menos	Entre 200 milissegundos e 500 milissegundos	Acima de 500 milissegundos
CLS	0,1 ou menos	Entre 0,1 e 0,25	Acima de 0,25

Fonte: adaptado de Walton (2020).

A análise dessas métricas é realizada, prioritariamente, em ambiente real (campo), com dados coletados a partir das interações dos usuários em diferentes dispositivos, redes e condições de uso. No entanto, muitas delas também podem ser medidas em laboratório, onde os testes são conduzidos de forma controlada e visam auxiliar os desenvolvedores a identificar e resolver antecipadamente eventuais problemas de desempenho (WALTON, 2020).

As ferramentas recomendadas para mensurar as métricas do *Core Web Vitals* incluem *PageSpeed Insights*, *Lighthouse*, *Chrome DevTools*, *Search Console* e a extensão *Web Vitals* (CHROME, 2025).

2.2 Trabalhos Relacionados

2.2.1 *Microfrontend: Um Estudo Sobre O Conceito E Aplicação No Frontend*

O estudo realizado por Nascimento e Sotto (2020) teve como objetivo demonstrar o conceito e a aplicação da arquitetura de MFE, com foco na divisão do *front-end* em partes menores e independentes. Além disso, busca evidenciar os benefícios e desafios da adoção dessa abordagem, bem como discutir os impactos resultantes da evolução dos sistemas de informação.

Para a realização do estudo, os autores conduziram uma pesquisa bibliográfica e desenvolveram uma aplicação com abordagem de MFE utilizando a biblioteca React.js e os frameworks Angular.js e Vue.js. O sistema implementado permite buscar e listar repositórios e

usuários por meio de uma API (*Application Programming Interface*) disponibilizada pelo GitHub.

Os resultados indicaram que a arquitetura de MFE pode reduzir a complexidade associada a aplicações monolíticas, diminuir o acoplamento e melhorar a escalabilidade e a organização do código. Além disso, os autores destacam que a abordagem proporciona maior flexibilidade no desenvolvimento de soluções empresariais. No entanto, ressaltam que a adoção de MFE deve ser feita de forma criteriosa, uma vez que seu uso inadequado em aplicações de pequeno porte pode gerar mais problemas do que benefícios.

Por fim, concluíram que uso da arquitetura de MFE permite avanços no desenvolvimento de *software* para a *web*, oferecendo flexibilidade e escalabilidade em sistemas distribuídos. No entanto, requer atenção à maturidade e à escolha adequada para evitar complexidade desnecessária e garantir a uniformidade entre os sistemas. Quando bem implementada, a arquitetura de MFE apresenta-se como uma solução vantajosa. O artigo destaca como a aplicação prática demonstrou seu funcionamento e benefícios.

2.2.2 Motivations, benefits, and issues for adopting Micro-Frontends: A Multivocal Literature Review

Taibi, Mezzalira e Peltonen (2021) conduziram um estudo motivado pela hesitação das empresas em adotar a arquitetura de MFE. O objetivo foi mapear e sintetizar, por meio de uma revisão literária multivocal (*Multivocal Literature Review* – MLR), o conhecimento disponível sobre MFE, evidenciando as motivações para sua adoção, os benefícios alcançados e os desafios enfrentados. Para isso, os autores elaboraram três questões de pesquisa: “Por que os profissionais estão adotando Microfrontends?”, “Quais são os benefícios obtidos com o uso de Microfrontends?” e “Os Microfrontends apresentam algum problema?”. A análise foi baseada em 173 fontes literárias, das quais apenas 43 foram consideradas relevantes para o estudo.

Os resultados indicaram que a principal motivação para a adoção de MFE é a busca por maior independência das equipes e a redução da complexidade de grandes aplicações *front-end* monolíticas. No que se refere aos benefícios, os autores destacaram a criação de equipes multifuncionais autônomas e a facilidade em escalar processos. Por outro lado, os desafios associados à adoção de MFE incluem o aumento do tamanho do *payload* (em português, carga útil) dos aplicativos, a duplicação de código, o acoplamento entre equipes e a complexidade no monitoramento das aplicações.

Os autores concluíram que o estudo atingiu seu objetivo ao destacar implicações relevantes da adoção de MFE para profissionais e pesquisadores da área. No entanto, ressaltaram a necessidade de investigações futuras mais amplas e detalhadas sobre os tópicos abordados.

2.2.3 Micro Frontend Application Shell and Module Federation Architecture implementation and comparison

O estudo elaborado por Bahiense-Junior et al. (2024) seguiu a abordagem *Design Science Research* (DSR) aplicada à Engenharia de *Software*. Para isso, os autores desenvolveram um modelo visual que captura as contribuições da pesquisa em três aspectos: teoria, prática e avaliação. Além disso, o estudo comparou as abordagens *Application Shell* e *Module Federation*, analisando tanto aspectos funcionais quanto não funcionais.

Os resultados obtidos evidenciaram que o *Application Shell* requer maior atenção em relação a segurança, roteamento e gerenciamento de estado. Apesar de facilitar a centralização de dependências, o *Application Shell* exige um controle de versão mais rigoroso. Além disso, ele é capaz de garantir a disponibilidade e atualização assíncrona de seus pacotes via CDN (*Content Delivery Network*). No entanto, pode enfrentar desafios de sincronização de estados caso os MFE sejam construídos em tecnologias diferentes, tornando necessária a adoção de eventos customizados.

Eles também apontaram que o *Module Federation* permite o carregamento dinâmico dos módulos remotos, porém, demanda avançado conhecimento em Webpack e um bom controle das versões compartilhadas. Cada projeto possui sua própria esteira de publicação (*deployment pipeline*), facilitando a escalabilidade e modularização. Embora promova um compartilhamento descomplicado de módulos e dependências, isso requer uma maior atenção em relação a integridade dos dados. Para estabelecer a comunicação em tempo real entre os MFE, recomenda-se o uso de eventos customizados ou do padrão *publish/subscribe*.

Os autores concluíram que a análise comparativa destacou que o *Application Shell* e o *Module Federation* são adequados para cenários distintos. Enquanto o *Application Shell* apresenta vantagens em projetos com múltiplas tecnologias e equipes distribuídas, o *Module Federation* é mais indicado para aplicações que exigem rápida escalabilidade e integração. Por fim, ressaltaram que a escolha entre as abordagens deve considerar as necessidades específicas do projeto e que, em estudos futuros, pretendem investigar a possibilidade de combinar ambas as soluções.

3 METODOLOGIA

Este estudo trata-se de uma pesquisa aplicada, com abordagem qualitativa e exploratória. O objetivo é levantar os desafios e estratégias da autenticação *Single Sign-On* em arquiteturas *Microfrontends* utilizando Vite e *Module Federation*. A pesquisa é baseada na revisão de literatura, experimentação prática e avaliação da aplicação.

Inicialmente, foi realizada uma pesquisa bibliográfica a fim de contextualizar *Microfrontends*, *Module Federation*, *Single Sign-On*, *cookies* HTTP e *Core Web Vitals* para embasamento. Em seguida, ocorreu o desenvolvimento de uma aplicação *showcase* a fim de oferecer um cenário realista para a experimentação e validação dos conceitos teóricos, funcionando como uma prova de conceito voltada para a compreensão dos benefícios, limitações e desempenho da arquitetura de MFE no ecossistema Vite com autenticação SSO.

3.1 Procedimentos de Implementação

A aplicação *showcase* foi desenvolvida utilizando a arquitetura de *Microfrontends*, integrando o Vite como ferramenta de *build* — processo de compilação de código — e o *framework* Vue.js 3 com suporte à linguagem TypeScript. Para viabilizar a composição entre as diferentes partes da aplicação, foi empregada a abordagem de *Module Federation* por meio do *plugin vite-plugin-federation*.

O projeto conta com um módulo responsável pela autenticação via SSO, que gera *tokens* de acesso seguindo o padrão de *cookies* HTTP. Essa abordagem permite a manutenção do estado de autenticação entre as diferentes partes da aplicação, facilitando o compartilhamento da sessão do usuário. Para simplificar a implementação e manter o foco na camada de *front-end*, foi utilizada a solução Auth0 (AUTH0, 2025) como provedor de identidade para o SSO. Além disso, foram desenvolvidos outros módulos que dependem da sessão autenticada para apresentar suas funcionalidades. A implementação seguiu uma sequência de passos definidos previamente (Quadro 3).

Quadro 3 – Etapas da Implementação da Arquitetura de *Microfrontends*.

Etapa	Descrição
Configuração do Ambiente	Configuração inicial das ferramentas e tecnologias utilizadas para o desenvolvimento da aplicação <i>showcase</i> .
Prototipação	Definição dos requisitos do sistema e criação do <i>design</i> da aplicação, contemplando aspectos funcionais e visuais.
Criação API de Autenticação	Projeto <i>back-end</i> que se comunica com o Auth0. Desenvolvido em Node.js e Express, com rotas de <i>login</i> , <i>logout</i> , registro de usuários e uso de <i>cookies</i> HTTP.
Criação MFE de Autenticação	Responsável por consumir a API de autenticação para realizar as ações necessárias na camada de <i>front-end</i> . Garante o compartilhamento do status de autenticação e dados do usuário autenticado com os demais módulos da aplicação.
Configuração <i>Module Federation</i>	Configuração que possibilitou a transformação dos projetos <i>front-end</i> em remotos, onde cada MFE expõem suas funcionalidades para o aplicativo hospedeiro.
Criação MFE Hospedeiro	Aplicativo hospedeiro que realiza a orquestração e a composição dos demais MFE, servindo como ponto de entrada principal da aplicação, integrando as funcionalidades provenientes dos MFE remotos, viabilizando a comunicação entre eles e garantindo a coesão da experiência do usuário final.
Criação MFE <i>Dashboard</i>	Aplicação remota que expõem uma página contendo um <i>dashboard</i> responsável pela exibição dos dados do usuário autenticado. É consumido pelo MFE Hospedeiro.
Criação MFE Conceituais	Módulos remotos de exemplo que expõem casos de uso relacionados a diferentes cenários de autenticação em MFE. Servem como referência para ilustrar comportamentos específicos no contexto de autenticação distribuída. São consumidos pelo MFE Hospedeiro.

Publicação	A estrutura de projetos foi hospedada na plataforma Cloudflare Pages (CLOUDFLARE, 2025). Para isso, cada MFE remoto foi disponibilizado em um subdomínio específico, enquanto o MFE hospedeiro foi configurado para atender pelo domínio principal. Para o <i>back-end</i> , foi utilizado o serviço Railway (RAILWAY, 2025), configurado em um subdomínio dentro do mesmo domínio utilizado pelos MFE.
------------	---

Fonte: elaborado pelos autores (2025).

3.2 Avaliação

Para validar a viabilidade da arquitetura proposta, foi realizada a análise dos aspectos de desempenho, complexidade de implementação e segurança.

O desempenho foi avaliado considerando o tempo de carregamento das páginas e a eficiência na composição dinâmica dos módulos. Utilizou-se a aba Performance e a aba *Lighthouse* do *Chrome DevTools* para coletar sistematicamente os valores dos *Core Web Vitals* (LCP, INP e CLS) de cada MFE remoto, integrado ao MFE Hospedeiro. Esses valores foram comparados com os parâmetros recomendados pela Google (Quadro 2), e discutidos quanto à sua aderência aos requisitos de experiência do usuário.

A complexidade foi analisada considerando a configuração do *Module Federation*, o gerenciamento de dependências compartilhadas e o impacto na modularização da aplicação. Essa avaliação permitiu identificar os desafios enfrentados e as decisões de configuração necessárias para garantir o correto funcionamento da arquitetura.

A segurança foi verificada por meio de inspeções manuais das implementações, validando o uso adequado de atributos de *cookies* HTTP e o correto compartilhamento de sessão entre os MFE. Embora não tenham sido realizados testes automatizados de vulnerabilidades, foram avaliadas práticas de segurança que visam mitigar riscos de vazamento de dados ou uso indevido de sessões.

3.3 Limitações da Pesquisa

O estudo limita-se à análise da adoção de autenticação via SSO em *Microfrontends*, considerando exclusivamente as tecnologias mencionadas. Dessa forma, não aborda o processo de autenticação no lado do *back-end*, nem aspectos aprofundados de segurança ou desafios de implementação para outros *frameworks* JavaScript e provedores de identidade.

4 DESENVOLVIMENTO

Este capítulo descreve o processo de desenvolvimento da aplicação proposta, detalhando o ambiente utilizado, as etapas seguidas para sua construção, os aspectos técnicos de sua arquitetura e modelagem, além de apresentar os principais artefatos gerados ao longo do projeto. A estrutura foi pensada para viabilizar a experimentação, considerando os desafios associados à integração da autenticação *Single Sign-On* em um cenário de *Microfrontends* utilizando Vue.js e Vite.

4.1 Ambiente de Desenvolvimento

O ambiente de desenvolvimento foi configurado com foco em modularidade, agilidade e integração contínua entre os MFE. Foi elaborada uma lista com os principais recursos utilizados (Quadro 4).

Quadro 4 – Tecnologias, Ferramentas e Serviços Utilizados.

Item	Tecnologia/Ferramenta/Serviço
Gerenciador de pacotes	Node.js (v22.14.0) com PNPM (<i>Performant Node Package Manager</i>)
Sistema operacional	Windows 11
Editor de código	WebStorm 2024.3.5
Controle de versão	Git, com repositórios hospedados no GitHub Classroom (MFECODIGO, 2025)
Autenticação SSO	Auth0
Aplicação <i>back-end</i>	Node.js e Express
Empacotador	Vite
<i>Module Federation</i>	<i>vite-plugin-federation</i> .
<i>Framework</i> JavaScript	Vue.js 3
<i>Design System</i>	Bulma CSS
Prototipação	Figma
Publicação <i>front-end</i>	Cloudflare Pages
Publicação <i>back-end</i>	Railway

Fonte: elaborado pelos autores (2025).

4.2 Etapas do Desenvolvimento

O processo de desenvolvimento foi conduzido de forma incremental e iterativa, permitindo validações frequentes e ajustes quando necessário. As etapas definidas (Quadro 2) foram seguidas de modo não linear, devido à demanda constante por alterações, correções e revisões que precisaram ser realizadas.

Com o ambiente e a prototipação prontos, a construção das aplicações seguiu uma ordem lógica de implementação (Quadro 5), que permitiu, inicialmente, o entendimento e a aplicação

da abordagem de *Module Federation*. Primeiramente, foi desenvolvido o MFE de autenticação (*mfe-auth*), responsável por fornecer a base de autenticação via SSO e servir como provedor de dados de autenticação para os demais módulos. Na sequência, foi criado o MFE hospedeiro (*mfe-host*), que possibilitou a integração dinâmica dos módulos por meio do consumo do *mfe-auth*, consolidando a prática de composição de MFE. Com essas duas bases estabelecidas, foi desenvolvido o MFE de dashboard (*mfe-dashboard*), que consome dados do *mfe-auth* e é integrado pelo *mfe-host*, demonstrando a orquestração entre módulos. A partir desse ponto, os demais MFE — *showcase* dos casos de autenticação (*mfe-auth-sc-with*, *mfe-auth-sc-half*, *mfe-auth-sc-another* e *mfe-auth-sc-without*) — foram implementados utilizando o mesmo modelo de autenticação e comunicação previamente definidos, seguindo uma estrutura padronizada da arquitetura.

Quadro 5 – Sequência de Desenvolvimento das Aplicações

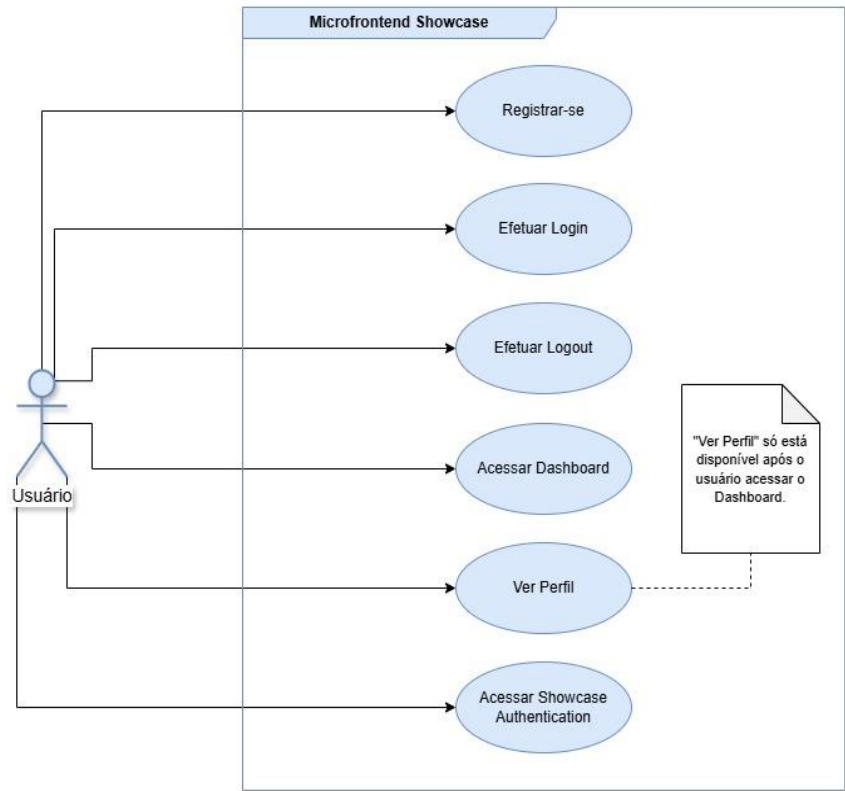
Aplicação	Descrição
<i>mfe-auth</i>	Forneceu a base de autenticação via SSO e foi essencial para disponibilizar informações de autenticação aos demais MFE.
<i>mfe-host</i>	Viabilizou a composição dinâmica dos módulos remotos, permitindo que o <i>mfe-auth</i> pudesse ser consumido e integrado corretamente.
<i>mfe-dashboard</i>	Depende diretamente dos dados de autenticação fornecidos pelo <i>mfe-auth</i> e é consumido pelo <i>mfe-host</i> .
<i>mfe-auth-sc-with</i> <i>mfe-auth-sc-half</i> <i>mfe-auth-sc-another</i> <i>mfe-auth-sc-without</i>	Foram favorecidos pela padronização da arquitetura advinda da construção prévia do <i>mfe-auth</i> e <i>mfe-dashboard</i> . Necessitam do status da autenticação fornecido pelo <i>mfe-auth</i> e são consumidos pelo <i>mfe-host</i> .

Fonte: elaborado pelos autores (2025).

4.3 Modelo Lógico da Aplicação

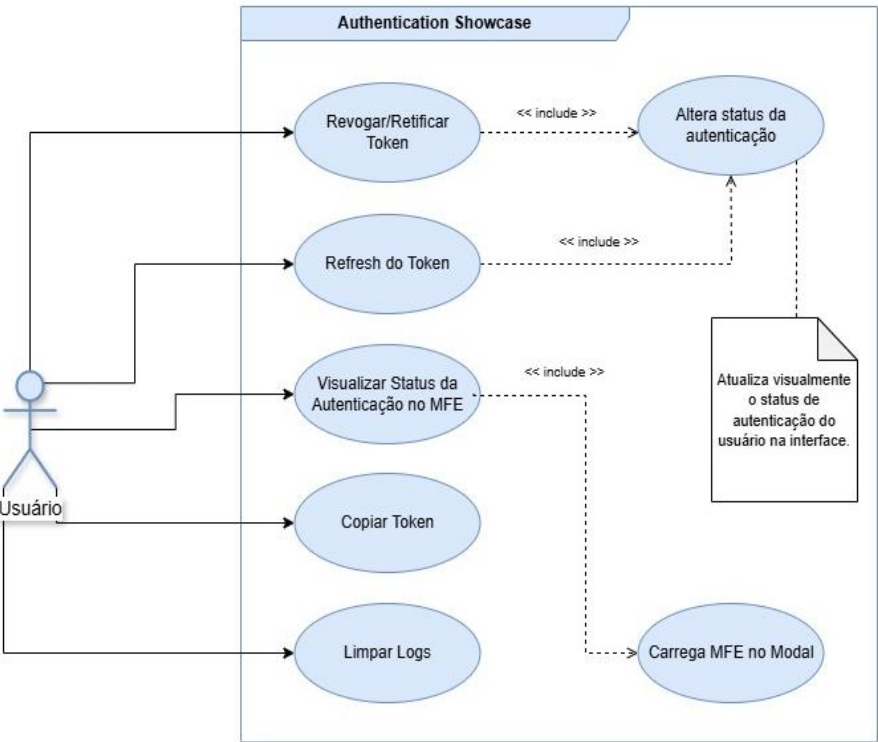
4.3.1 Diagrama de Casos de Uso

Diagrama 2 – Casos de Uso da aplicação *Microfrontend Showcase*.



Fonte: elaborado pelos autores (2025).

Diagrama 3 – Casos de Uso do domínio *Authentication Showcase*.



Fonte: elaborado pelos autores (2025).

4.3.2 Requisitos do Sistema

Diagrama 4 – Requisitos Funcionais do Sistema.

ID	Nome	Descrição	Prioridade	Critério de Aceitação
RF001	Cadastro de usuário	O sistema deve permitir o cadastro de usuários com nome, e-mail e senha.	Alta	O usuário conseguir cadastrar e fazer login.
RF002	<i>Login</i> de usuário	O sistema deve permitir o <i>login</i> de usuários através do e-mail e senha.	Alta	O usuário conseguir efetuar o <i>login</i> e acessar a plataforma.
RF003	<i>Logout</i> de usuário	O sistema deve permitir o <i>logout</i> de usuários ao clicar no botão de sair.	Alta	O usuário ser redirecionado ao formulário de <i>login</i> e perder acesso a plataforma.
RF004	Botão de usuário	O sistema deve ter um botão para exibir as informações de nome, e-mail e botão de <i>logout</i> .	Alta	O usuário pode clicar no botão e exibir um modal com as informações.
RF005	Menu lateral	O sistema deve exibir um menu lateral com todas as opções de rotas mais o botão de <i>logout</i> .	Alta	O usuário pode acessar as páginas e efetuar o <i>logout</i> através do menu.
RF006	Opções menu lateral	O sistema deve possuir um menu lateral com as opções Dashboard e <i>Authentication</i> .	Alta	O usuário ver e interagir com os menus.
RF007	Página dashboard	O sistema deve exibir a página do dashboard com informações da autenticação e dos <i>microfrontends</i> .	Alta	O usuário visualizar o dashboard com cartões informativos da autenticação e <i>microfrontends</i> .
RF008	Página <i>showcase</i> de <i>Authentication</i>	O sistema deve exibir a página com o <i>showcase</i> de <i>Authentication</i> .	Alta	O usuário conseguir visualizar e interagir com as funcionalidades do <i>showcase</i> da <i>Authentication</i> .
RF009	Revogar <i>token</i>	O sistema deve permitir testar o cenário de token revogado no <i>showcase</i> de <i>Authentication</i> .	Alta	O usuário pode revogar o <i>token</i> e gerar impacto nos <i>microfrontends</i> que precisam dele.
RF010	Retificar <i>token</i>	O sistema deve permitir ao usuário testar o cenário de retificar token no <i>showcase</i> de <i>Authentication</i> .	Alta	O usuário pode retificar o <i>token</i> e gerar impacto nos <i>microfrontends</i> que precisam dele.

RF011	<i>Refresh token</i>	O sistema deve permitir simular o comportamento de <i>refresh</i> do <i>token</i> no <i>showcase</i> de <i>Authentication</i> .	Alta	O usuário conseguir dar <i>refresh</i> no <i>token</i> e gerar impacto nos <i>microfrontends</i> que precisam dele.
RF012	Visualizar estado dos <i>microfrontends</i>	O sistema deve permitir interagir com os <i>microfrontends</i> e observar o comportamento deles de acordo com o status do <i>token</i> no <i>showcase</i> de <i>Authentication</i> .	Alta	O usuário visualizar o comportamento dos <i>microfrontends</i> diante do status do <i>token</i> .
RF013	Copiar o token	O sistema deve permitir copiar o token no <i>showcase</i> de <i>Authentication</i> .	Alta	O usuário conseguir copiar o <i>token</i> .
RF014	Descrição <i>showcase</i> de <i>Authentication</i>	O sistema deve exibir a descrição do <i>showcase</i> .	Alta	O usuário conseguir visualizar a descrição.

Fonte: elaborado pelos autores (2025).

Diagrama 5 – Requisitos Não Funcionais do Sistema.

ID	Nome	Categoria	Descrição	Critério de Aceitação
RNF001	Compatibilidade de Navegador	Portabilidade	O sistema deve ser compatível com os navegadores Chrome, Firefox e Safari.	Testes realizados nos três navegadores não apresentam falhas.
RNF002	Responsividade	Portabilidade	O sistema deve estar otimizado apenas para telas com <i>breakpoint</i> de 1280px.	Teste realizado aponta que o sistema está responsivo apenas para dispositivos <i>desktop</i> .

Fonte: elaborado pelos autores (2025).

4.3.3 Apresentação da Interface

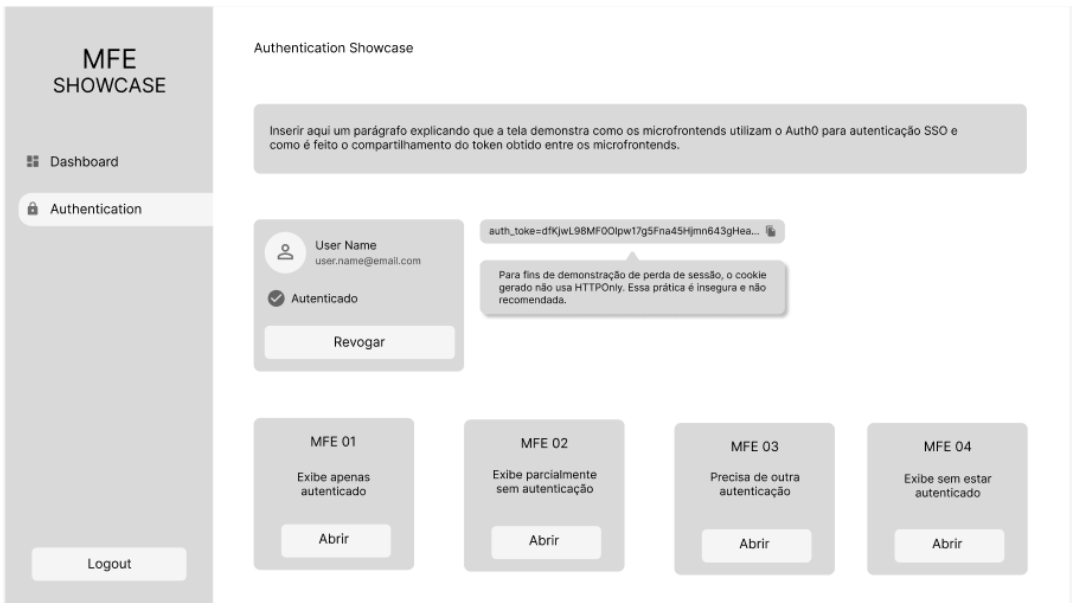
Para a construção da interface (MFEPROTOTIPO, 2025), foi utilizada a ferramenta de prototipação de *designs Figma* (FIGMA, 2025).

Figura 2 – Protótipo da interface do *Dashboard*.



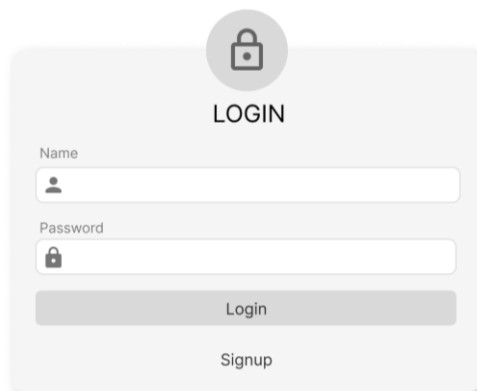
Fonte: elaborado pelos autores (2025).

Figura 3 – Protótipo da interface da página *Showcase Authentication*.



Fonte: elaborado pelos autores (2025).

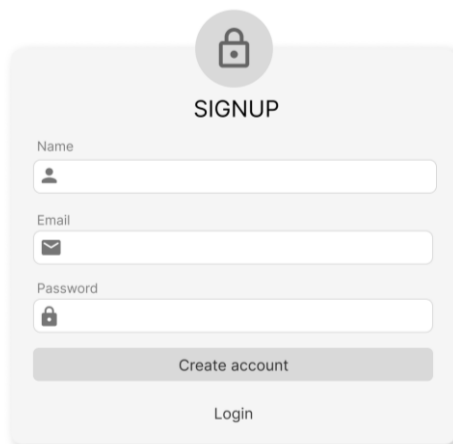
Figura 4 – Protótipo da interface da página de *Login* no formulário de autenticação.



O protótipo da interface de login apresenta um cartão centralizado com um ícone de cadeado no topo. Abaixo do ícone, o título "LOGIN" é exibido. O formulário contém dois campos de entrada: "Name" com um ícone de usuário e "Password" com um ícone de cadeado. Abaixo dos campos, há dois botões: "Login" e "Signup".

Fonte: elaborado pelos autores (2025).

Figura 5 – Protótipo da interface da página de *Login* no formulário de cadastro.



O protótipo da interface de login no formulário de cadastro apresenta um cartão centralizado com um ícone de cadeado no topo. Abaixo do ícone, o título "SIGNUP" é exibido. O formulário contém três campos de entrada: "Name" com um ícone de usuário, "Email" com um ícone de envelope e "Password" com um ícone de cadeado. Abaixo dos campos, há dois botões: "Create account" e "Login".

Fonte: elaborado pelos autores (2025).

5 RESULTADOS

Conforme estabelecido, aspectos de desempenho, complexidade de implementação e segurança foram analisados. A seguir, são descritas as observações para cada critério.

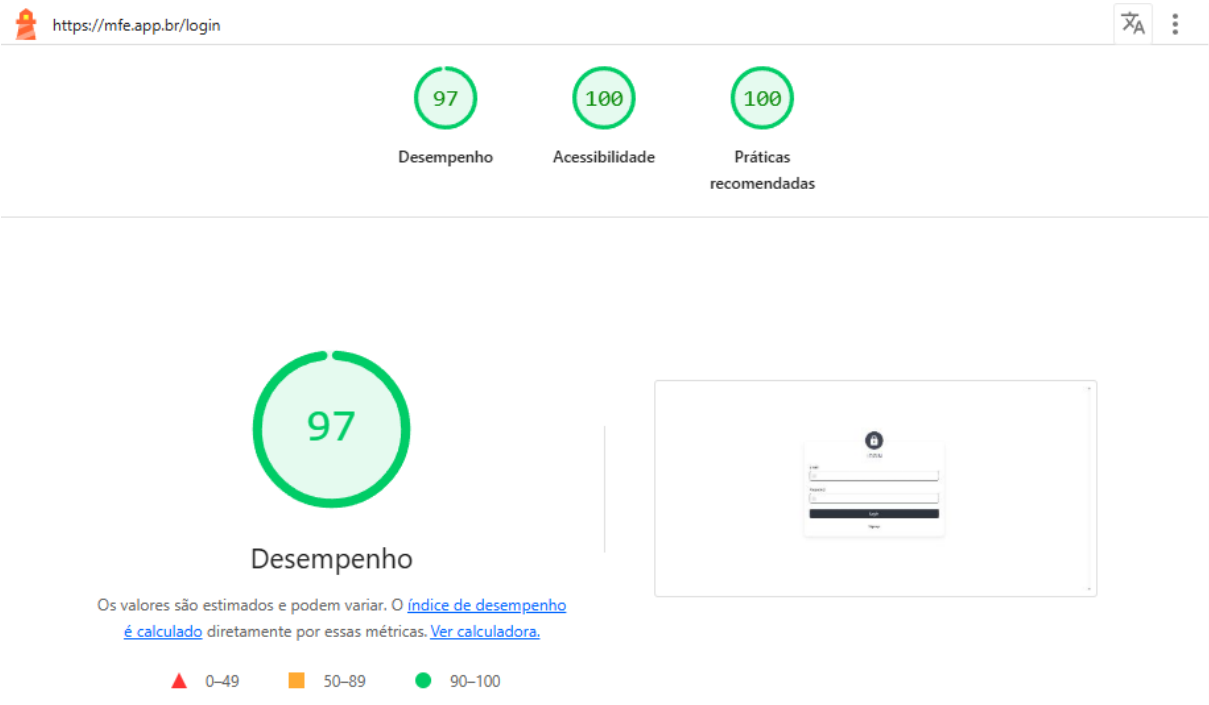
5.1 Desempenho

Para analisar o desempenho da aplicação, foi necessário realizar a publicação do sistema (MFESHOWCASE, 2025), de forma a obter dados mais representativos do ambiente de produção. Em ambiente de desenvolvimento local, as métricas poderiam ser distorcidas por fatores como a ausência de minificação de código, uso de bibliotecas de apoio e configurações de depuração, impactando negativamente a avaliação.

A ferramenta *Lighthouse* foi utilizada para gerar um relatório (MFECODIGO, 2025) de performance geral (Figura 6). No ambiente de produção, a aplicação hospedada no domínio mfe.app.br apresentou uma pontuação geral de 97 pontos em desempenho, 100 em acessibilidade e 100 em melhores práticas. Esses resultados evidenciam que a aplicação está

otimizada para carregamento eficiente e acessibilidade, além de seguir boas práticas de desenvolvimento.

Figura 6 – Relatório gerado pelo *Lighthouse* da aplicação (MFESHOWCASE, 2025).

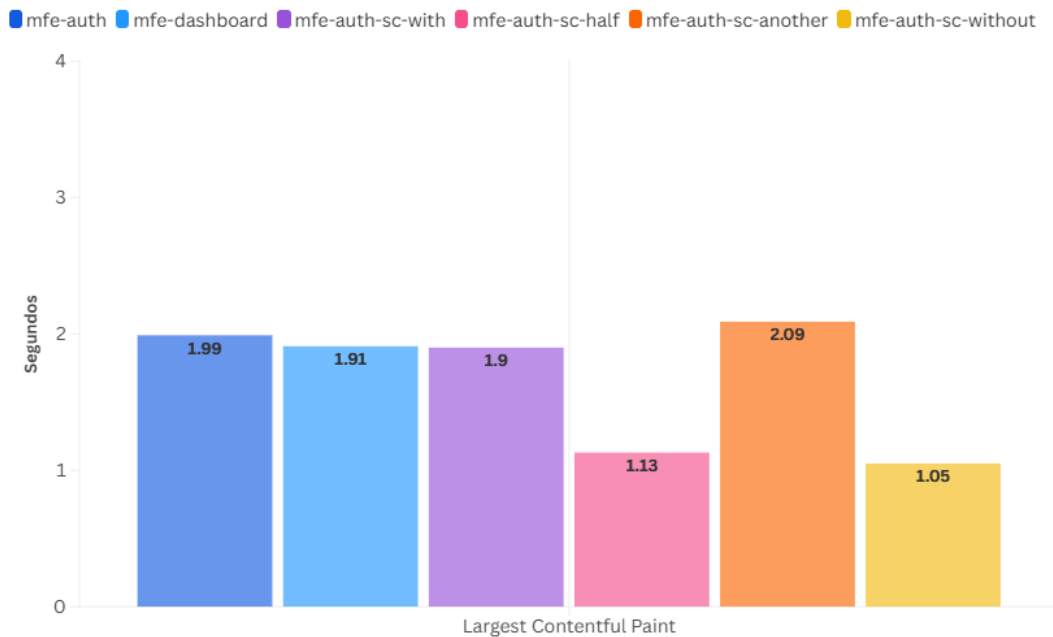


Fonte: *Chrome DevTools Lighthouse* (2025).

A análise comparativa — elaborada a partir dos relatórios (MFECODIGO, 2025) gerados pela aba performance da ferramenta *Chrome DevTools* — entre os diferentes *Microfrontends* desenvolvidos (Gráfico 1) revela que, de modo geral, os tempos de FCP permanecem próximos de 1 a 2 segundos e os tempos do LCP ficam entre 27 e 44 milissegundos. Indicando um bom tempo de resposta para a maioria dos módulos, esses valores estão alinhados com as melhores práticas recomendadas para aplicações *web* (Quadro 2), evidenciando que o uso do *Module Federation* e o carregamento assíncrono foram implementados de forma eficaz, garantindo uma experiência de usuário fluida.

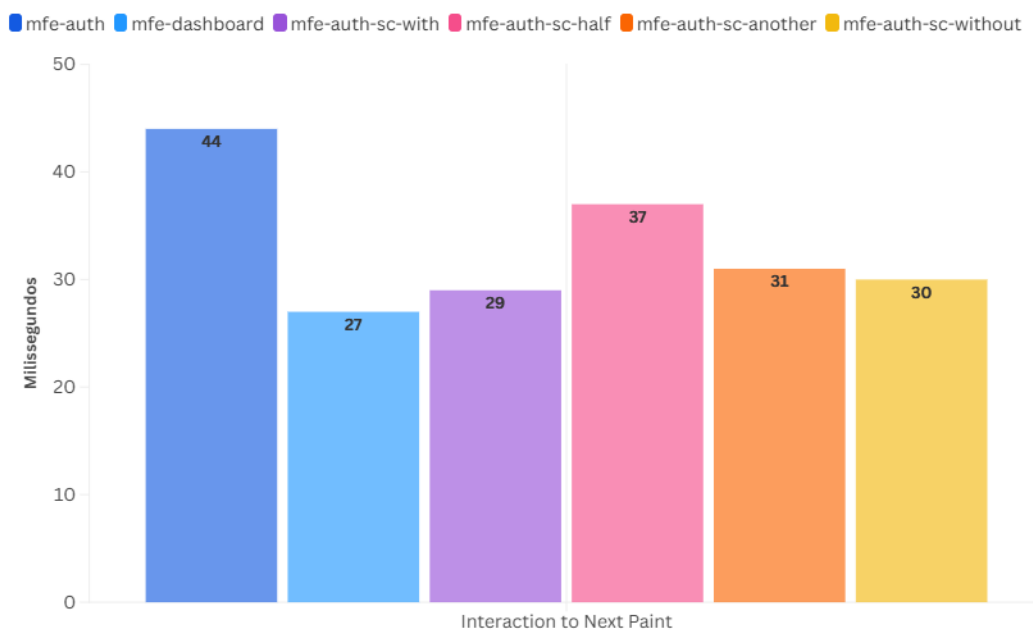
Além disso, os relatórios mostraram que o CLS permaneceu em zero para todos os módulos, demonstrando a estabilidade do *layout* e a ausência de deslocamentos inesperados que poderiam comprometer a experiência do usuário (Quadro 2). Esse resultado reforça a maturidade da arquitetura e a atenção dada aos detalhes de usabilidade.

Gráfico 1 – Gráfico da métrica LCP de cada MFE remoto carregado dentro do *mfe-host*.



Fonte: elaborado pelos autores (2025).

Gráfico 2 – Gráfico da métrica INP de cada MFE remoto carregado dentro do *mfe-host*.



Fonte: elaborado pelos autores (2025).

De forma geral, a análise evidencia que a arquitetura de *Microfrontends*, aliada ao uso de carregamento assíncrono e otimizações de desempenho, resultou em uma aplicação rápida, estável e alinhada às boas práticas de desenvolvimento *web* moderno.

5.2 Complexidade de Implementação

Durante o desenvolvimento e configuração do ambiente, foram enfrentados diversos desafios, entre eles: (i) a definição e configuração dos módulos remotos para expor seus ativos corretamente; (ii) a configuração do módulo hospedeiro para consumir os ativos dos remotos via URLs externas; (iii) a execução de todos os MFE em modo HTTPS (*HyperText Transfer Protocol Secure*) (HTTPS, 2025) localmente, requisito para o funcionamento adequado com *cookies* HTTP e segurança entre domínios; (iv) as estratégias de compartilhamento de dados entre os MFE, incluindo informações de usuário; (v) a garantia do compartilhamento de dependências comuns evitando conflitos de versão; (vi) a criação de novos MFE a partir de uma estrutura comum — o que evidencia a importância de manter um *template* base com arquitetura e configurações padronizadas; e (vii) a execução local de todos os MFE, uma vez que o *Module Federation* opera sobre arquivos já construídos, exigindo que cada MFE remoto seja construído e iniciado em modo preview para funcionar corretamente no ambiente de desenvolvimento.

A respeito da configuração do *Module Federation*, observa-se que, de modo geral, ela não apresenta grandes desafios, pois segue uma estrutura relativamente intuitiva e objetiva. Nessa configuração, é possível declarar quais recursos serão expostos (*exposes*), quais serão consumidos de outros módulos (*remotes*) e quais dependências deverão ser compartilhadas (*shared*) entre os módulos participantes (Figura 7 e Figura 8).

Figura 7 – Trecho do código de configuração do *Module Federation* no mfe-auth.

```
export default defineConfig({
  plugins: [
    federation({
      name: 'auth',
      filename: 'remoteEntry.js',
      exposes: {
        './LoginView': './src/views/LoginView.vue',
        './LogoutButton': './src/components/buttons/LogoutButton.vue',
      },
      remotes: {},
      shared: [
        {
          vue: {
            singleton: true,
            requiredVersion: '^3.5.13',
          },
          axios: {
            requiredVersion: '^1.8.4',
          },
          bulma: {
            singleton: true,
            eager: false,
          },
        },
      ],
    }),
  ],
})
```

Fonte: elaborado pelos autores (2025).

Figura 8 – Trecho do código de configuração do *Module Federation* no mfe-host.

```
export default defineConfig({
  plugins: [
    federation({
      name: 'host',
      filename: 'remoteEntry.js',
      remotes: {
        auth: 'https://auth.mfe.app.br/assets/remoteEntry.js',
        dashboard: 'https://dashboard.mfe.app.br/assets/remoteEntry.js',
        'with-auth': 'https://auth.with.mfe.app.br/assets/remoteEntry.js',
        'half-auth': 'https://auth.half.mfe.app.br/assets/remoteEntry.js',
        'without-auth': 'https://auth.without.mfe.app.br/assets/remoteEntry.js',
      },
      shared: [
        {
          vue: {
            singleton: true,
            requiredVersion: '^3.5.13',
          },
          axios: {
            requiredVersion: '^1.8.4',
          },
          bulma: {
            singleton: true,
            eager: false,
          },
        },
      ],
    },
  ],
})
```

Fonte: elaborado pelos autores (2025).

Adicionalmente, para viabilizar a integração entre os MFE remotos e a aplicação hospedeira, foi necessário realizar ajustes específicos na configuração do Vite em cada módulo remoto (Figura 9). Essa etapa foi fundamental para assegurar que o hospedeiro pudesse consumir os recursos disponibilizados pelos remotos sem restrições de acesso ou problemas de comunicação. Embora a configuração siga uma estrutura relativamente simples, ela exigiu conhecimento técnico aprofundado sobre o Vite, especialmente no que se refere à gestão de CORS (*Cross-Origin Resource Sharing*) (CORS, 2025), resultando em uma curva de aprendizado média.

Figura 9 – Trecho do código de configuração do Vite para viabilizar o consumo dos MFE.

```
export default defineConfig({
  server: {
    cors: {
      origin: '*',
      methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'OPTIONS'],
      allowedHeaders: ['X-Requested-With', 'content-type', 'Authorization'],
    },
    headers: {
      'Access-Control-Allow-Origin': '*',
      'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
      'Access-Control-Allow-Headers':
        'Origin, X-Requested-With, Content-Type, Accept, Authorization',
      'Access-Control-Allow-Credentials': 'true',
    },
  },
})
```

Fonte: elaborado pelos autores (2025).

Outro ponto importante relacionado à complexidade de implementação foi a etapa de publicação dos módulos no Cloudflare Pages (MFESHOWCASE, 2025). Foi necessário

configurar o domínio principal (mfe.app.br) e atribuir subdomínios para cada MFE, garantindo o isolamento e a integração correta entre eles. Para o mfe-auth-sc-another, não foi configurado um subdomínio, pois era essencial para o caso de uso que ele estivesse em outro domínio. A configuração de DNS (*Domain Name System*) (DNS, 2025) para apontar cada subdomínio para o respectivo projeto exigiu atenção especial para evitar conflitos e garantir o funcionamento adequado das rotas. Para o processo de publicação, utilizou-se a estratégia de empacotamento com Vite, gerando o diretório de código publicável (*dist*) para cada MFE e, em seguida, atribuindo cada *dist* ao respectivo projeto no Cloudflare Pages, o que facilitou a publicação contínua e a atualização independente dos módulos.

Apesar da complexidade inicial, uma vez estabelecida a estrutura base, a adição e integração de novos MFE tornaram-se tarefas mais simples. A modularidade da arquitetura proporcionou maior independência entre os projetos, ainda que o custo de iniciar todos os MFE localmente represente um desafio prático.

5.3 Segurança

A utilização de *cookies* HTTP como estratégia para SSO mostrou-se mais segura que o uso de *tokens* em *localStorage* ou *sessionStorage*. Isso porque evita a exposição direta das credenciais no navegador.

Algumas boas práticas adotadas durante a implementação contribuíram para reforçar a segurança da aplicação, entre elas: (i) utilização de HTTPS com *cookies* configurados com *SameSite=None* e *Secure=true* (HTTPCOOKIES, 2025), garantindo integridade e confidencialidade nas sessões; (ii) implementação de validação de origem e controle de CORS nas chamadas HTTP; (iii) separação clara entre o MFE responsável pela autenticação (mfe-auth) e os MFE consumidores, evitando vazamentos de contexto sensível; (iv) ausência de exposição de *tokens* JSON *Web Token* (JWT) no lado do cliente.

Apesar das medidas adotadas, para fins de comprovação da existência do *cookie* na aplicação *showcase*, a configuração *httpOnly* (HTTPCOOKIES, 2025) dos *cookies* foi desabilitada — uma prática que não é recomendada em ambientes reais, pois compromete a proteção contra acessos indevidos via *scripts* maliciosos no navegador.

5.4 Análise dos Resultados e Discussão

Com a aplicação *showcase* desenvolvida, foi possível constatar que o login único, proporcionado pelo SSO, representa uma alternativa eficaz para sistemas *front-end* distribuídos. A utilização de *cookies* HTTP seguros, configurados para um ou mais domínios, simplificou a autenticação entre os diferentes *Microfrontends* e garantiu maior segurança nas requisições feitas para a API. Essa abordagem evita a exposição de *tokens* no lado do cliente (navegador), reduzindo a superfície de ataque e contribuindo para uma arquitetura mais segura e coesa.

Também foi observado que a adoção do *Module Federation* no ecossistema Vite mostrou-se uma alternativa viável para a construção de *Microfrontends*. A composição entre os MFE remotos e o MFE hospedeiro funcionou de forma satisfatória, permitindo a exposição e o consumo dinâmico dos módulos de maneira eficiente. No entanto, assim como apontado por Bahiense-Junior et al. (2024) ao analisar o uso da *Module Federation* no Webpack, a implementação dessa arquitetura no Vite também exige conhecimentos aprofundados da ferramenta, além da definição cuidadosa de estratégias para o compartilhamento de dados. Complementando a recomendação dos autores sobre o uso de eventos customizados e do padrão *publish/subscribe*, foi possível constatar, com base na aplicação desenvolvida, que o uso de um estado global compartilhado também se mostrou eficaz para apoiar a comunicação em tempo real entre os MFE.

Além disso, as análises de desempenho realizadas evidenciaram que a aplicação *showcase* apresentou tempos de carregamento satisfatórios e uma experiência de uso fluida, confirmando que a composição dinâmica via *Module Federation* não gerou impactos negativos para o usuário final.

Outro ponto importante é a complexidade de gerenciar múltiplos projetos em execução simultânea no ambiente local. Em contextos reais, é fundamental definir domínios, recursos compartilhados e estratégias de comunicação para garantir o desacoplamento entre os *Microfrontends*. Uma estratégia para reduzir a dependência entre projetos é a utilização de objetos simulados (*mocks*) durante o desenvolvimento, permitindo implementar funcionalidades de forma isolada.

Apesar das vantagens, a arquitetura de *Microfrontends* não é uma solução universal para todos os problemas de escalabilidade. Ela requer planejamento cuidadoso, definição clara das responsabilidades de cada módulo e atenção às interdependências. A modularização excessiva pode aumentar a complexidade, dificultar o gerenciamento de dependências e tornar o processo de desenvolvimento mais oneroso.

Portanto, a adoção de *Microfrontends* deve ser feita de forma deliberada e alinhada aos objetivos do projeto, avaliando se seus benefícios superam os desafios de implementação. Essa abordagem é mais indicada para aplicações de larga escala, com múltiplas equipes trabalhando em paralelo ou em casos em que há necessidade de independência entre módulos e atualização contínua. Já para projetos menores ou com equipes reduzidas, a complexidade adicional pode não se justificar frente aos benefícios esperados.

6 CONCLUSÃO

O presente trabalho teve como objetivo investigar a integração da autenticação *Single Sign-On* em arquiteturas de *Microfrontends* utilizando Vue.js, Vite e *Module Federation*. A construção da aplicação *showcase* possibilitou explorar, de forma prática, os desafios técnicos e as estratégias necessárias para viabilizar a comunicação segura e eficiente entre módulos isolados.

Os resultados evidenciaram que o uso de *cookies* HTTP foi eficaz para manter sessões seguras entre domínios, reduzindo a exposição de dados sensíveis no cliente. Além disso, a aplicação do *Module Federation* mostrou-se adequada para a composição dinâmica dos módulos, embora tenha demandado conhecimento técnico aprofundado para lidar com o compartilhamento de dependências e a configuração de múltiplos projetos em execução simultânea. As análises de desempenho realizadas confirmaram que a arquitetura proposta apresentou tempos de carregamento satisfatórios, alinhando-se às melhores práticas de desenvolvimento *web*.

A arquitetura modular proposta demonstrou benefícios importantes, como maior flexibilidade para a evolução das aplicações e uma estrutura que favorece o isolamento e a autonomia dos módulos. Entretanto, observou-se que a adoção dessa abordagem requer planejamento cuidadoso, especialmente em relação à orquestração e à comunicação entre os módulos, para garantir a coesão e a escalabilidade do sistema. Estratégias como o uso de objetos simulados podem apoiar o desenvolvimento local, reduzindo a dependência de múltiplos projetos em execução simultânea.

Como trabalhos futuros, recomenda-se a ampliação da investigação em torno de estratégias de autorização baseadas em escopo, bem como o estudo de soluções que simplifiquem a orquestração e a implantação dos projetos, permitindo melhores análises e maior eficiência no desenvolvimento. Além disso, sugere-se a realização de testes de desempenho com dados de campo para validar o comportamento da aplicação em cenários reais de uso, bem como a execução de testes de segurança automatizados, a fim de identificar e mitigar eventuais vulnerabilidades.

REFERÊNCIAS

NEWMAN, Sam. **Building Microservices**. Sebastopol, California: O'Reilly Media, Inc., 2015. E-book. Disponível em: <https://oreilly.com/library/view/building-microservices/9781491950340>.

MEZZALIRA, Luca. **Building Micro-Frontends**. Sebastopol, California: O'Reilly Media, Inc., 2021. E-book. Disponível em: <https://www.oreilly.com/library/view/building-micro-frontends/9781492082989>.

GEERS, Michael. **Micro Frontends in Action**. 2. ed. Shelter Island, Nova York: Manning Publications Co., 2020. E-book. Disponível em: <https://www.oreilly.com/library/view/micro-frontends-in/9781617296871>.

KHONONOV, Vlad. **Learning Domain-Driven Design**. Sebastopol, California: O'Reilly Media, Inc., 2021. E-book. Disponível em: <https://learning.oreilly.com/library/view/learning-domain-driven-design/9781098100124/>.

WEBPACK. **Module Federation**. Disponível em: <https://webpack.js.org/concepts/module-federation>. Acesso em: 23 de fev. 2025.

VUE. **Frequently Asked Questions**. Disponível em: <https://vuejs.org/about/faq.html>. Acesso em: 23 de fev. 2025.

NASCIMENTO, Charles Henrique Paiva do; SOTTO, Eder Carlos Salazar. MICROFRONTEND: um estudo sobre o conceito e aplicação no frontend. **Interface Tecnológica**, Online, v. 17, p. 153–165, 2020. Disponível em: <https://revista.fatectq.edu.br/index.php/interfacetecnologica/article/view/798/458>.

TAIBI, Davide; MEZZALIRA, Luca; PELTONEN, Severi. Motivations, Benefits, and Issues for Adopting Micro-Frontends: A Multivocal Literature Review. **Information and Software Technology**, Online, v. 136, p. 106571, 2021. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0950584921000549>.

BAHIENSE-JUNIOR, Marcos Roberto G.; SANTOS, Lanier M.; BEZERRA R, Renan; BEZERRA C, Erick. Micro Frontend Application Shell and Module Federation Architecture Implementation and Comparison. **IEEE**, Online, p. 166–171, 2024. Disponível em: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10829917>.

SCAPICCHIO, Mark; FORREST, Amber. **O que é Single Sign On (SSO)?**. Disponível em: <https://www.ibm.com/br-pt/topics/single-sign-on>. **IBM**, Online, 2024. Acesso em: 04 de abr. 2025.

VITE. **Vite**. Disponível em: <https://vite.dev>. Acesso em: 08 abr. 2025.

VITEFEDERATION. **vite-plugin-federation**. Disponível em: <https://github.com/originjs/vite-plugin-federation>. Acesso em: 08 abr. 2025.

AUTH0. **Auth0**. Disponível em: <https://auth0.com>. Acesso em: 08 abr. 2025.

HTTPCOOKIES. **Using HTTP cookies**. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies>. Acesso em: 15 mai. 2025.

CLOUDFLARE. **Cloudflare Pages**. Disponível em: <https://pages.cloudflare.com/>. Acesso em: 16 mai. 2025.

RAILWAY. **Railway**. Disponível em: <https://railway.com/>. Acesso em: 16 mai. 2025.

CHROME. **Documentation**. Disponível em: <https://developer.chrome.com/docs>. Acesso em: 15 mai. 2025.

WALTON, Philip. Web Vitals. **web.dev**, Online, 2020. Disponível em: <https://web.dev/articles/vitals?hl=pt-br>. Acesso em: 30 de mai. 2025.

POWELL, Phill; SMALLEY, Ian. **O que é arquitetura monolítica?**. Disponível em: <https://www.ibm.com/think/topics/monolithic-architecture>. **IBM**, Online, 2024. Acesso em: 30 de mai. 2025.

HTTP. **Overview of HTTP**. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>. Acesso em: 30 de mai. 2025.

HTTPS. **HTTPS**. Disponível em: <https://developer.mozilla.org/en-US/docs/Glossary/HTTPS>. Acesso em: 30 de mai. 2025.

CORS. **Cross-Origin Resource Sharing (CORS)**. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/CORS>. Acesso em: 30 de mai. 2025.

DNS. **O que é DNS?**. Disponível em: <https://www.cloudflare.com/pt-br/learning/dns/what-is-dns/>. Acesso em: 30 de mai. 2025.

MFEPROTOTIPO. **MFE Protótipo.** Disponível em:
<https://www.figma.com/design/5TRUnDpR4s7FrOwGNsm1h5/MFE-Showcase?node-id=0-1&t=OTTtSfLRkrO2iKOT-1>.

MFESHOWCASE. **MFE Showcase.** Disponível em: <https://mfe.app.br>.

MFECODIGO. **MFE Código.** Disponível em: <https://github.com/ICEI-PUC-Minas-PBR-SI/pbr-si-2025-1-p8-tcc-t1-30104-pbr-si-2025-1-p8-mfe>.